

Major Project Report on

Rule-Based Self-Healing System for Enhancing the Reliability of Web Applications



in partial fulfillment for the award of the degree of
Bachelor of Information Technology Engineering

Submitted By

Aayush Gnawali	[22120105]
Basanta Paudel	[22120114]
Dipendra Niroula	[22120121]
Purshottam Chand Bohara	[22120132]

Submitted To

Department of Software and IT Engineering
Everest Engineering College
Sanepa-2, Lalitpur
July, 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project supervisor **Er.Santosh Bist** sir and faculty members of the Department of Software/Information Technology for their continuous guidance, encouragement, and valuable suggestions throughout the course of this project. Their insights and academic support played a crucial role in shaping the direction and quality of our work.

We are also thankful to our friends and classmates for their cooperation, discussions, and moral support during the development of this project. Finally, we would like to acknowledge our families for their patience, motivation, and encouragement, which enabled us to complete this project successfully.

ABSTRACT

Modern UI changes frequently break Selenium locators, causing false test failures and demanding heavy manual maintenance from quality assurance teams. To solve this, this project introduces a rule-based autonomous self-healing framework that intercepts locator failures and restores test execution in real time. During successful runs, the framework records a "Golden Fingerprint" for each element—capturing its ID, visible text, XPath, CSS class, tag name, and structural relationships. When a failure occurs, it evaluates live DOM elements against the stored fingerprint using four weighted heuristic rules, applying a confidence score to determine whether to heal automatically, heal cautiously with a review flag, or request manual intervention.

The framework operates strictly on the test execution side, leaving both the application source code and existing test scripts completely untouched. Its fully deterministic decision logic provides predictable, auditable recovery actions, distinguishing it from machine-learning alternatives. All self-healing events, confidence scores, logs, and infrastructure health metrics are displayed on a real-time dashboard, ultimately delivering a measurable reduction in test maintenance effort and build failures without requiring structural application changes.

Keywords: *Autonomous recovery, Autonomic computing, Fault detection, Heuristic scoring, Rule-based management, Self-healing systems, Web application reliability.*

TABLE of CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT.....	ii
LIST OF FIGURES	vi
LIST OF TABLES	vii
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Statement of Problem.....	1
1.3 Objective	2
1.3.1 General Objective	2
1.3.2 Specific Objective	2
1.4 Scope.....	3
1.4.1 System Boundary and Constraints	4
1.5 Applications	5
1.5.1 QA Teams in Small and Medium Software Projects	5
1.5.2 Continuous Integration Pipelines.....	5
1.5.3 Site and Operations Teams	5
1.5.4 Educational and Research Use	5
CHAPTER 2: LITERATURE REVIEW	6
2.1 Rule-Based System Monitoring	6
2.2 Autonomic Computing and Self-Healing Systems	6
2.3 Self-Healing Web Applications	7
2.4 Failure Detection Using Predefined Rules.....	7
2.5 Automated Recovery and Corrective Actions	7
2.6 Event Logging and Administrator Notification	7
2.7 Limitations of Existing Work	Error! Bookmark not defined.
2.8 Research Gap and Project Relevance	8
CHAPTER 3: METHODOLOGY	10
3.1 System Design	10
3.1.1 Architectural Block Diagram	10
3.1.2 Five Layered Architecture	12
3.2 Data Modeling -Entity-Relationship Design	15
3.2.1 ER-Diagram	15

3.3 Heuristic Scoring and Decision Logic	16
3.3.1 Logic Flowchart	16
3.3.2 Scoring Algorithm Table	18
3.4 System Operational Workflow	20
3.4.1 Operational Lifecycle.....	20
3.4.2 Learning Mode Workflow	21
3.4.3 Healing Mode Workflow	23
3.4.4 Feedback and Update Cycle.....	26
3.4.5 Integration with an Existing Test Suite.....	27
3.5 Software Development Life Cycle.....	28
3.5.1 Phases of Development.....	29
3.5.2 Why Incremental Model Fits Better	30
3.6 Technology Stack.....	30
3.7 The Real: Time Dashboard	31
3.8 Testing & Evaluation	33
3.9 Hardware and Software Requirements	34
CHAPTER 4: RESULT AND ANALYSIS.....	35
4.1 Evaluation Setup	35
4.2 Locator: Healing Results.....	36
4.2.1 Baseline versus Self: Healed Execution	36
4.2.2 Confidence Scoring and Rule Contributions	37
4.2.3 Isolated Front:End Change Scenarios.....	38
4.2.4 Permanent Source:Level Self:Correction	40
4.2.5 Intent:Aware Healing.....	41
4.2.6 Baseline Recovery After Fingerprint Loss	41
4.3 Safety Behaviour: Manual:Intervention Halt.....	42
4.4 Infrastructure: Healing Results	43
4.5 Browser:Side Anomaly Detection	43
4.6 Automated Verification Harness.....	44
4.7 Consolidated Results.....	45
4.8 Analysis.....	47
4.9 Comparison with Traditional Selenium Testing	50
4.10 Summary of Findings.....	51
CHAPTER 5: CONCLUSION AND FUTURE ENHANCEMENT	52

5.1 Conclusion	52
5.2 Future Enhancement	53
5.2.1 Limitations and Challenges Encountered	53
5.2.2 Proposed Improvements.....	54
5.2.3 Additional Features and Modules	55
5.2.4 Scalability, Integration and Wider Application	55
References.....	57

LIST OF FIGURES

Figure 3.1: Architectural Block Diagram of Self-Healing System.....	11
Figure 3.2: ER-Diagram of Self-Healing System	15
Figure 3.3: Logic Flowchart of Self-Healing System.....	17
Figure 3.4: Learning Mode Workflow	22
Figure 3.5: Healing Mode Flowchart.....	25
Figure 3.6: Close looped feedback and update cycle.....	26
Figure 3.7: Incremental SDLC Model of Self:Healing System.....	28
Figure 4.1: Dashboard UI Heuristic Healing tab, showing each heal with its confidence score and R1–R4 breakdown	38
Figure 4.2: Dashboard Locator Self:Correction tab, showing the healed locator written back to the automation source	41
Figure 4.3: Dashboard Alert Notification Logs tab, showing a safety:gate refusal recorded as a critical entry	42
Figure 4.4: Dashboard Server Infrastructure Heals tab, showing a rule:based recovery action and its trigger metrics.....	43

LIST OF TABLES

Table 3.1: Heuristic Scoring Algorithm Table	18
Table 4.1: Baseline versus Self: Healed Execution of the Same Test Suite	36
Table 4.2: Confidence Scores and Rule Contributions for Each Healed Element.....	37
Table 4.3: Recovery Outcomes Across Isolated Front:End Change Scenarios	38
Table 4.4: Automated Verification Harness Results.....	44
Table 4.5: Demonstrated Outcomes Across All Three Fault Domains	45
Table 4.6: Fulfilment of the Specific Objectives	48
Table 4.7: Comparison with Traditional Selenium Testing.....	50

CHAPTER 1: INTRODUCTION

1.1 Background

Automated testing has become an essential part of modern web application development. Selenium-based test suites are used to verify application behavior continuously, and organizations depend on them to detect regressions before release. However, the reliability of these suites depends on a fragile assumption: that the identifiers used to locate user-interface elements remain stable over time.

In practice this assumption fails constantly. During ordinary development a developer may rename an element identifier, change a CSS class as part of a redesign, alter a name attribute, or restructure the DOM hierarchy. None of these changes affect a human user, and the application continues to behave correctly. The automated test suite, however, collapses: Selenium raises a `NoSuchElementException`, tests fail, and the build is reported as broken. This phenomenon is commonly referred to as locator rot.

The cost of locator rot is twofold. First, engineering time is consumed identifying which locators broke and updating them by hand, an activity that produces no new test coverage. Second, failing runs are significantly slower than passing runs, because every unresolved locator consumes its full explicit-wait timeout before failing. Repeated false failures also erode confidence in the test suite, leading teams to ignore or disable tests.

Self-healing test automation addresses this problem by allowing the framework to recover from locator failures at runtime. Rather than terminating on the first missing element, the system identifies the element the script intended to reach by comparing the live page against a previously recorded reference model, and allows execution to continue. Commercial tools that provide this capability generally rely on machine-learning models, which are costly, require external services, and cannot explain why a particular element was selected. For small and medium-scale projects, a deterministic rule-based approach offers the same practical benefit with predictable behavior, no external dependencies, and full explainability.

1.2 Statement of Problem

Quality assurance engineers write Selenium test scripts against the state of an application at the time the script is authored. The locators embedded in those scripts are static, while the application they target changes continuously. This produces a recurring and predictable failure pattern:

1. A developer modifies the markup of the application, for example by renaming an element identifier or CSS class.
2. The existing test script, which has not changed, can no longer locate the affected element.
3. Selenium raises a `NoSuchElementException` and the test fails, despite the application functioning correctly.
4. A quality assurance engineer must manually inspect the page, identify the new locator, and edit the test script.

Three constraints make this problem difficult to solve in practice. First, quality assurance engineers cannot modify the application source code, so the failure cannot be resolved by changing the markup back. Second, a naive automatic recovery mechanism is dangerous, because selecting the wrong element causes the test to pass while validating incorrect behavior, which is worse than a visible failure. Third, any recovery decision must be explainable, since a quality assurance team cannot accept a repair, it is unable to justify or audit.

Existing approaches either ignore the problem, leaving teams to absorb the maintenance cost, or apply opaque machine-learning models that cannot state why a substitution was made. There is therefore a need for a deterministic, rule-based framework that detects locator failures at runtime, identifies the intended element through explainable heuristic scoring, refuses to act when its confidence is too low, allows the unchanged test script to continue executing, and reports each decision to the quality assurance team.

1.3 Objective

1.3.1 General Objective

The general objective of this project is to design and implement a rule-based autonomous self-healing framework that restores the execution of Selenium test scripts when their user-interface locators break, by comparing the live application against a previously recorded Golden Fingerprint and applying weighted heuristic rules, without modifying the application under test and without requiring any change to the existing test scripts.

1.3.2 Specific Objective

To achieve the general objective, the following specific objectives are defined:

- To develop a Learning Mode that records a Golden Fingerprint of every element a test script successfully locates during a passing test run, capturing element

identifier, visible text, XPath, CSS class, tag name, and neighboring element relationships.

- To develop a Healing Mode that intercepts locator failures at runtime and identifies the intended element by scoring live DOM candidates against the stored fingerprint using four weighted heuristic rules.
- To derive a repaired locator from the identifying attributes of the matched live element and report it to the quality assurance team as an actionable recommendation.
- To repair the execution of the test script without modifying the source code, markup, or behavior of the application under test.
- To apply confidence-based decision thresholds so that the system heals automatically only when the match is reliable, flags uncertain matches for review, and refuses to act when the match is too weak to be safe.
- To provide a real-time dashboard reporting the original locator, the repaired locator, the heuristic score and its rule breakdown, the reason for the repair, the confidence level, the log history, and recommendations for the quality assurance team.

1.4 Scope

The scope of this project centers on UI locator healing for Selenium test automation as the flagship, most thoroughly designed and evaluated capability, and extends the same rule-based framework to two supporting domains: server-side infrastructure healing and browser-side anomaly detection. The framework operates as an add-on layer around an existing application and test suite it does not replace Selenium, write new test cases, or redesign the application under test.

Concretely, the framework:

- Monitors test execution for locator-resolution failures, scores candidate elements using four heuristic rules (text, XPath, CSS class, and neighboring-element similarity), and repairs the locator including rewriting the fix back into the source test script when confidence is high enough.
- Monitors server-side health signals and applies predefined recovery actions when a fault pattern is detected, logging every action taken.
- Captures browser-side resource anomalies (such as a failed image or asset load) invisible to server-side monitoring, and raises them as alerts.

- Logs every detection and healing attempt across all three domains to one unified dashboard.
- Escalates low-confidence or unrecognized cases to a human reviewer instead of guessing.

1.4.1 System Boundary and Constraints

The framework operates strictly on the automation side of the testing process. The application under test is treated as a read-only target: its HTML, CSS, JavaScript, and server-side source code are never inspected for modification and are never altered by the system. This boundary is a deliberate design constraint rather than a limitation, and it reflects the operating reality of quality assurance teams, who do not have authority to modify production application code in order to make a test pass.

Recovery is therefore applied at two levels only. At the execution level, the framework returns the correct live element to the running test script so that execution continues without interruption. At the metadata level, the framework updates its own locator repository so that subsequent runs are aware of the change. Any correction to the test script itself is presented to the quality assurance engineer as a reviewable recommendation rather than being applied silently.

The framework is scoped to Selenium-based browser automation of web applications rendered in Chrome and Firefox. It addresses failures caused by changes to element identifiers, CSS classes, attribute values, visible text, and DOM structure. It does not address failures caused by genuine defects in the application, by removed functionality, or by changes in business logic; in those cases, the test is expected to fail, and the framework's safety threshold is designed to ensure that it does.

1.5 Applications

The proposed system is intended for use in environments where high availability and reliability of web applications are required, but where complex and resource-intensive management solutions are impractical. It is particularly suitable for small to medium-scale web applications operating in controlled settings.

Because the framework spans three related reliability domains behind one dashboard, it is useful to more than one role on an engineering team.

1.5.1 QA Teams in Small and Medium Software Projects

Small and medium engineering teams often maintain UI test suites without dedicated automation engineers. For these teams, the locator-healing engine reduces the day-to-day burden of chasing locator failures after routine UI changes, freeing QA time for actual test design and defect investigation.

1.5.2 Continuous Integration Pipelines

In CI pipelines where UI tests run on every merge, locator breakage caused by an unrelated front-end change can block unrelated pull requests. The framework can reduce these false-failure blockages by healing recoverable locators automatically before a build is marked as failed.

1.5.3 Site and Operations Teams

The infrastructure-healing engine gives operations staff a lightweight first line of defense against common, well-understood fault patterns, reducing time-to-recovery for issues that do not require deep investigation, while still logging every automated action for later review.

1.5.4 Educational and Research Use

Because all healing decisions in every domain follow explicit, inspectable rules rather than a trained model, the framework is also well suited to academic study of rule-based self-healing systems it is straightforward to trace exactly why a given healing decision was made, which supports both classroom use and further research extensions.

CHAPTER 2: LITERATURE REVIEW

The concept of self-healing systems has been widely studied to improve the reliability and availability of software systems. Self-healing systems aim to automatically detect failures and recover from them without human intervention. This chapter reviews existing research related to rule-based monitoring, failure detection, autonomic computing, and self-healing mechanisms in web and software systems. It also highlights the limitations of existing approaches and positions the proposed project within this research area.

2.1 Rule-Based System Monitoring

Rule-based monitoring systems use predefined rules to observe system behavior and detect abnormal conditions. Sloman [1] introduced early concepts of policy- and rule-driven system management, emphasizing that predefined rules can control system behavior without modifying core application logic. Such systems are suitable for environments where failure conditions are known in advance.

Further studies show that rule-based monitoring is effective for detecting specific failures such as service unavailability or threshold violations [2]. However, most rule-based systems require manual intervention after detection, which increases downtime. This limitation motivates the need for automated corrective actions.

2.2 Autonomic Computing and Self-Healing Systems

The idea of autonomic computing was formally introduced by Kephart and Chess [3], where systems are designed to manage themselves using monitoring, analysis, and corrective actions. One key characteristic of autonomic systems is self-healing, which focuses on automatic fault detection and recovery.

Research indicates that autonomic systems can significantly reduce system downtime by responding immediately to failures [4]. However, many autonomic computing frameworks are complex and resource-intensive, making them unsuitable for lightweight web applications. This highlights the need for a simpler autonomic layer based on predefined rules.

2.3 Self-Healing Web Applications

Several studies have focused on self-healing mechanisms specifically for web applications. Garlan et al. [5] proposed architecture-based self-repair techniques where systems recover using predefined strategies. While effective, such approaches often require tight integration with application architecture.

Other research emphasizes monitoring web services to detect failures such as service crashes or performance degradation [6]. These approaches often focus only on detection and alerting, leaving recovery to administrators. This gap supports the development of systems that both detect failures and automatically execute recovery actions.

2.4 Failure Detection Using Predefined Rules

Failure detection based on predefined rules is a practical approach for systems with known failure patterns. Studies show that threshold-based and rule-based detection methods are simpler and more predictable compared to learning-based methods [7]. Such approaches are easier to implement and validate, especially in academic and small-scale projects.

However, existing systems often lack flexibility in defining recovery actions, or they focus solely on infrastructure-level failures rather than application-level issues [8].

2.5 Automated Recovery and Corrective Actions

Automated corrective actions are essential for reducing application downtime. Research by Salehie and Tahvildari [9] highlights that recovery mechanisms such as service restart, reconfiguration, or fallback actions can restore normal operations quickly when triggered automatically.

Many existing solutions focus on large-scale systems and assume advanced orchestration support. This makes them unsuitable for lightweight environments. The proposed project instead focuses on simple corrective actions triggered directly by rule evaluation.

2.6 Event Logging and Administrator Notification

Logging and notification are critical components of management systems. Studies emphasize that logging helps administrators analyze past failures and improve system reliability [10]. Notification mechanisms ensure that administrators remain informed even when recovery is automated.

However, many monitoring tools generate excessive alerts without performing recovery, leading to alert fatigue. This reinforces the importance of combining logging and notification with autonomous healing.

2.7 Research Gap and Project Relevance

The reviewed literature highlights the significance of self-healing mechanisms and rule-based management in improving system reliability and availability. Existing studies demonstrate that automated monitoring, failure detection, and recovery actions can effectively reduce system downtime [3], [5], [9]. However, most of the existing research focuses on complex autonomic frameworks, large-scale distributed systems, or cloud-native environments, which may not be suitable for small or standalone web applications [4], [8].

A clear research gap exists in the development of simple and lightweight self-healing systems that can be applied directly to individual web applications without requiring extensive infrastructure support or architectural modifications. Many existing approaches emphasize advanced orchestration, learning-based techniques, or tightly coupled recovery mechanisms, which increase system complexity and limit their practical use in academic or small-scale settings.

Based on the analysis of existing work, the following research gaps are identified:

1. Limited availability of lightweight self-healing solutions specifically designed for single web applications.
2. Over-reliance on complex autonomic frameworks that are difficult to implement and maintain in simple environments.
3. Insufficient focus on rule-based recovery mechanisms that offer predictability and ease of validation.
4. Lack of integrated systems that combine monitoring, failure detection, recovery, logging, and notification within a single management layer.
5. Minimal emphasis on student-level or educational implementations, where clarity and simplicity are essential.

The proposed project addresses these gaps by developing a rule-based autonomous web application management system that continuously monitors a running web application and its services.

Unlike complex autonomic frameworks discussed in existing literature, the proposed system prioritizes simplicity, transparency, and practical implementation. By focusing on a controlled and lightweight environment, this project demonstrates how established self-healing principles can be effectively applied to improve the reliability of web applications within the scope of a student major project.

CHAPTER 3: METHODOLOGY

This chapter describes the systematic approach followed to develop the Rule-Based Self-Healing Framework. The development process was focused on creating an autonomous system that can detect UI failures, analyze the changes using weighted heuristics, and self-correct without human intervention unless the confidence score is critically low.

3.1 System Design

The framework is designed with a layered architecture to separate the execution of automation scripts from the intelligent recovery logic.

3.1.1 Architectural Block Diagram

The system architecture presents a layered view of the proposed Rule-Based Self-Healing Web Application System, clearly separating application execution, monitoring, decision-making, and recovery mechanisms. This layered design improves clarity, modularity, and aligns the system behavior with the project objective of autonomous fault detection and recovery.

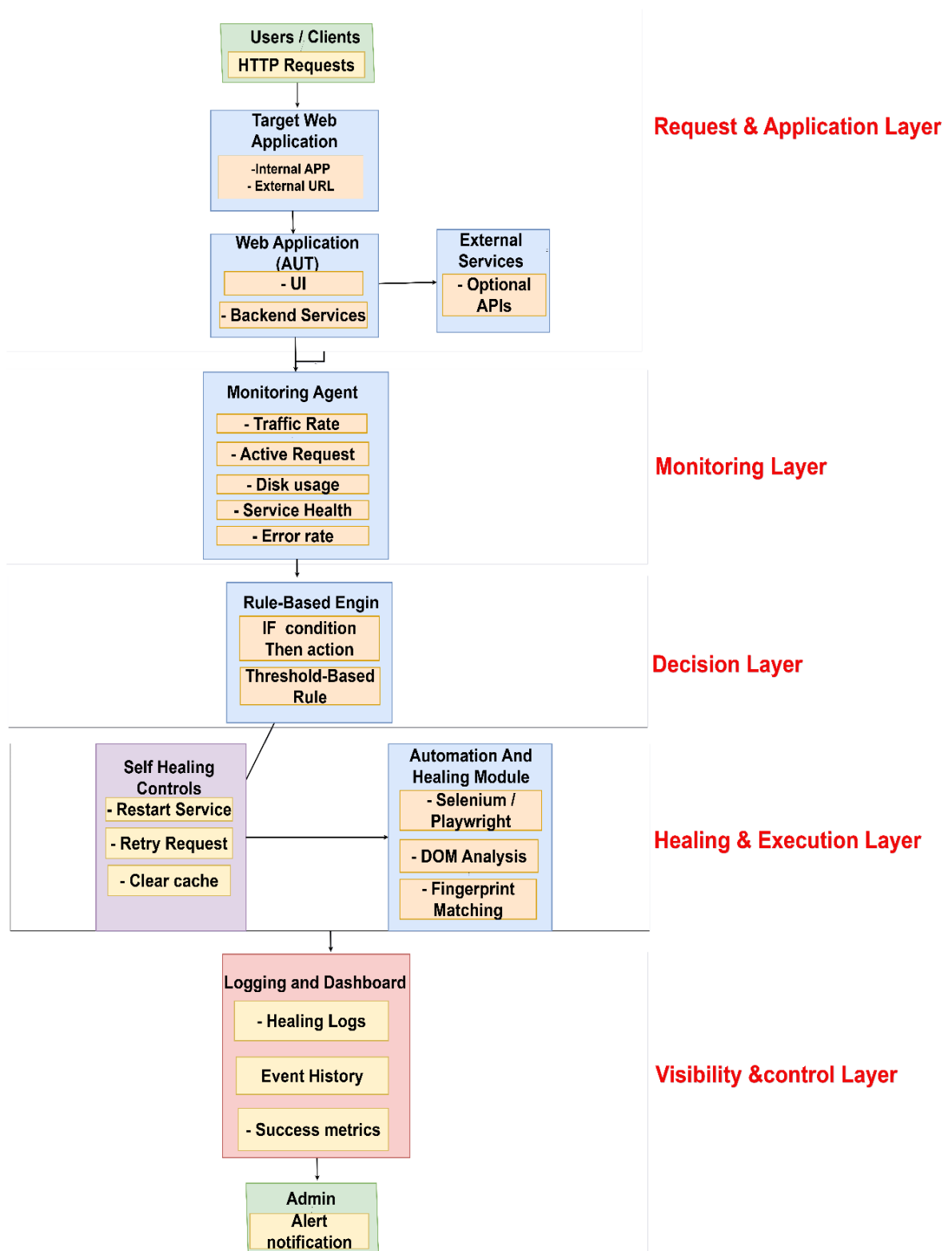


Figure 3.1: Architectural Block Diagram of Self-Healing System

3.1.2 Five Layered Architecture

3.1.2.1 Request & Application Layer

The Request & Application Layer represents the primary operational environment of the proposed self-healing system. Users or clients initiate HTTP requests that are directed toward the Target Web Application. The target application may operate using either internal APIs or external URLs depending on the deployment environment and application architecture.

The Web Application, also referred to as the Application Under Test (AUT), consists of two major components:

1. User Interface (UI)
2. Backend Services

The UI manages user interaction and presentation, while the backend services process requests, execute business logic, and communicate with external systems. The application may also interact with optional external services or third-party APIs to provide additional functionality such as authentication, payment processing, or data retrieval.

This layer represents the normal execution flow of the system and serves as the environment where runtime failures, performance degradation, or service interruptions may occur. Maintaining the availability and stability of this layer is the primary objective of the self-healing framework.

Within this layer the application under test is treated strictly as an observation target. The self-healing framework reads the rendered Document Object Model in order to extract candidate elements, but it does not modify the application's markup, stylesheets, scripts, or backend source at any point. This separation is essential to the design: the application is the environment in which failures are observed, not the component that is repaired. All corrective action taken by the framework occurs in the Healing and Execution Layer and affects only the automation script's execution and the locator metadata repository.

3.1.2.2 Monitoring Layer

The Monitoring Layer continuously observes the runtime behavior and operational health of the application. A dedicated Monitoring Agent collects and analyzes system metrics from the target application and associated services in real time.

The monitoring process focuses on the following parameters:

1. Traffic rate
2. Active request count
3. Disk usage

4. Service health
5. Error rate

The monitoring agent operates passively without interrupting normal application execution. Its primary responsibility is to provide accurate and up-to-date runtime information to the Decision Layer for fault detection and recovery analysis.

This layer acts as the foundation of the self-healing mechanism because all recovery decisions depend on the quality and accuracy of the collected monitoring data.

3.1.2.3 Decision Layer

The Decision Layer is responsible for identifying abnormal system conditions and determining the appropriate recovery action. This layer is implemented using a Rule-Based Engine that evaluates monitored metrics against predefined threshold-based conditions.

The engine follows deterministic IF–THEN logic, where specific runtime conditions trigger predefined healing actions. For example:

- High traffic rate may indicate system overload
- Excessive error rates may indicate service failure
- Unhealthy service status may indicate backend instability
- High disk usage may indicate resource exhaustion

When such conditions exceed configured thresholds, the rule engine activates the corresponding recovery mechanism.

The rule-based approach ensures:

1. Predictable system behavior
2. Easier validation and testing
3. Controlled automated recovery
4. Reduced complexity compared to adaptive AI-based systems

This design is suitable for controlled environments where reliability and interpretability are essential.

3.1.2.4 Healing & Execution Layer

The Healing & Execution Layer performs automated recovery actions whenever failures or abnormal conditions are detected by the Decision Layer.

This layer consists of two major components:

Self-Healing Controls

The Self-Healing Controls execute predefined recovery operations aimed at restoring normal system functionality. These operations include:

- Restarting failed services

- Retrying failed requests
- Clearing cache

These actions are designed to minimize downtime while avoiding full system restarts whenever possible.

Automation and Healing Module

The Automation and Healing Module handles application-level automation failures and UI-related issues. This module utilizes automation frameworks such as Selenium or Playwright to perform:

- Re-execution of failed automation tasks
- DOM analysis
- Fingerprint matching

DOM analysis helps identify structural changes in the web interface, while fingerprint matching assists in locating modified or relocated UI elements. This enables the system to recover from predictable UI changes and automation failures without manual intervention. After recovery actions are executed, the resulting system state is sent back to the Monitoring Layer through a feedback mechanism. This allows the system to verify whether the healing process was successful and ensures continuous operational validation.

3.1.2.5 Visibility & Control Layer

The Visibility & Control Layer provides monitoring transparency, system reporting, and administrative oversight for the self-healing framework.

This layer contains the Logging and Dashboard module, which records:

1. Healing logs
2. Event history
3. Success metrics

These records help administrators analyze system behavior, evaluate healing effectiveness, and identify recurring issues.

The layer also includes an Admin Alert Notification mechanism. If a failure cannot be resolved automatically or if the recovery confidence level falls below acceptable thresholds, the system generates alerts for administrative review and manual intervention.

3.2 Data Modeling -Entity-Relationship Design

3.2.1 ER-Diagram

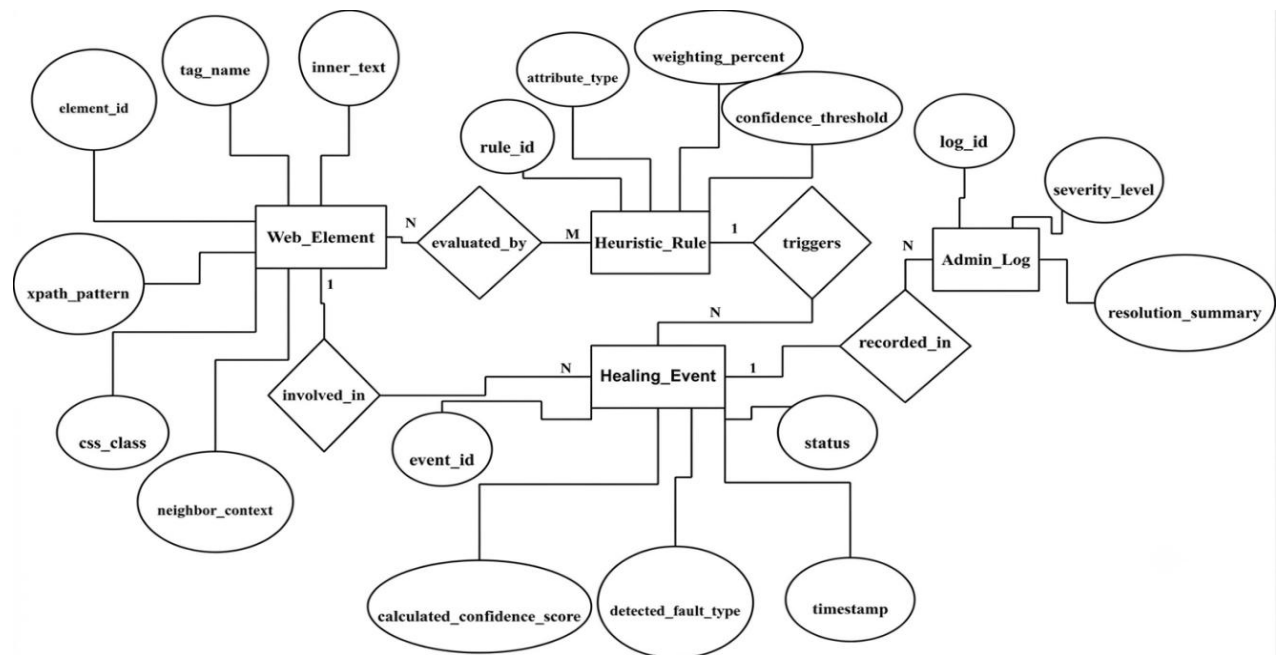


Figure 3.2: ER-Diagram of Self-Healing System

ER Diagram Description: Self-Healing Architecture

The entity-relationship diagram models the data required to record a baseline during a passing test run, to evaluate a locator failure during a later run, and to trace every autonomous decision back to the test that triggered it.

1. Application

Represents the application under test. Attributes: app_id, app_name, base_url, build_version, baseline_recorded_at. An application contains (1: N) many Web_Elements. This entity allows the framework to maintain independent baselines for different applications and different builds.

2. Test_Script

Represents a Selenium test authored by the quality assurance team. Attributes: script_id, file_path, test_name. A Test_Script uses (N:M) many Web_Elements, and a Web_Element may be used by many Test_Scripts. This relationship is what allows Learning Mode to record only the elements the tests actually depend upon.

3. Web_Element (The Target)

Represents one user-interface element and its Golden Fingerprint. Attributes: `element_id`, `logical_name`, `tag_name`, `inner_text`, `xpath_pattern`, `css_class`, `neighbor_signature`, `captured_at`. A `Web_Element` belongs_to (N:1) an application.

4. Locator

Represents a means of addressing a `Web_Element`. Attributes: `locator_id`, `strategy` (`id`, `css`, `xpath`, `name`, `link text`), `value`, `status` (`original`, `broken`, `repaired`), `stability_score`. A `Web_Element` is_addressed_by (1: N) many `Locators`, which allows the original and the repaired locator to coexist and be compared.

5. Heuristic_Rule

Represents one scoring rule. Attributes: `rule_id`, `attribute_type`, `weighting_percent`, `confidence_threshold`. There is a Many-to-Many (N:M) relationship labelled `evaluated_by` between `Web_Element` and `Heuristic_Rule`, indicating that multiple rules are applied to a single element and that a single rule evaluates many elements.

6. Test_Run

Represents one execution of a test suite. Attributes: `run_id`, `started_at`, `browser`, `mode` (`learning` or `healing`), `result`. A `Test_Run` produces (1: N) many `Healing_Events`. This entity supplies the execution context that the dashboard reports.

7. Healing_Event

Captures one recovery attempt. Attributes: `event_id`, `timestamp`, `detected_fault_type`, `original_locator`, `repaired_locator`, `calculated_confidence_score`, `rule_breakdown`, `decision` (`automatic`, `cautious`, `halted`), `reason`, `status`. `Healing_Event` is linked to `Web_Element` by a One-to-Many (1: N) relationship named `involved_in`, and to `Test_Run` by `produced_by`.

8. Admin_Log

Provides traceability for administrative review. `Healing_Event` is recorded_in (1: N) the `Admin_Log`. Attributes: `log_id`, `severity_level`, `resolution_summary`, `recommendation`.

3.3 Heuristic Scoring and Decision Logic

3.3.1 Logic Flowchart

The flowchart defines the decision-making path. It includes the “Learning Mode” for initial data capture and the “Healing Mode” for error recovery.

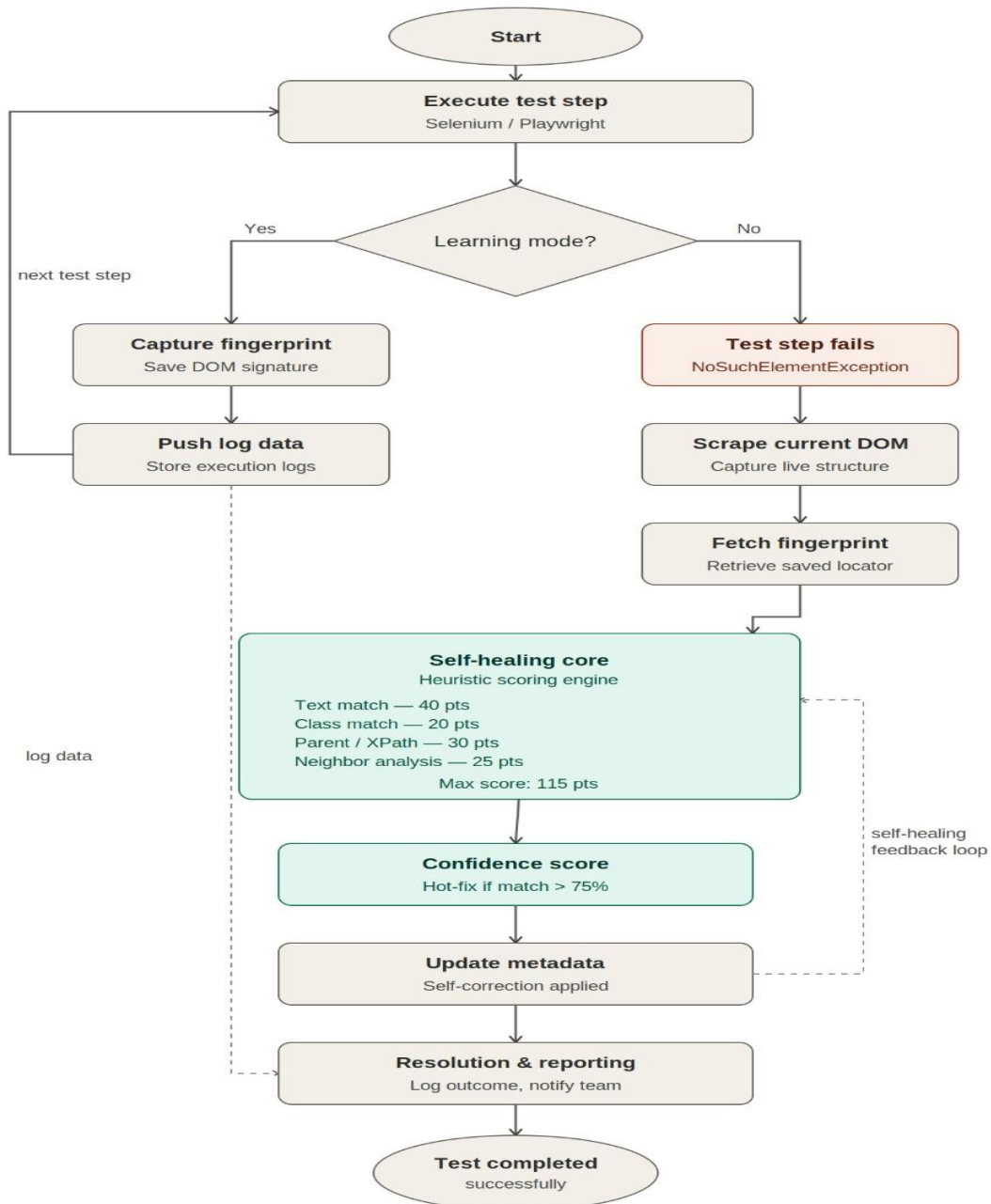


Figure 3.3: Logic Flowchart of Self-Healing System

Step 1: Baseline Check. At the start of a run the system checks whether a Golden Fingerprint baseline exists for the application under test. If no baseline exists, the system enters Learning Mode and records one from the current, known-good state of the application.

Step 2: Normal Execution and Baseline Capture. The test script executes normally. Each time a locator resolves successfully, the corresponding element's fingerprint is captured or

refreshed. A baseline is recorded only from a passing run, since learning from a broken page would record the fault as the reference state.

Step 3: Failure Detection. During a later run, if a locator fails to resolve, the system intercepts the resulting `NoSuchElementException` and extracts the live DOM instead of allowing the test to terminate.

Step 4: Scoring Engine. Every candidate element in the live DOM is compared against the Golden Fingerprint of the element the script intended to reach, using the weighted rules R1 to R4 defined in Table 3.1.

Step 5: Threshold Decision. The composite confidence score determines the action: at or above 75 per cent the system heals automatically; between 20 and 75 per cent it heals cautiously and flags the event for review; below 20 per cent it halts and raises an alert rather than risk interacting with the wrong element.

Step 6: Repaired Locator Derivation. For fying attributes of the winning live elementary extracted and ranked by stability, and the highest-ranked attribute is expressed as the repaired locator.

Step 7: Locator Update and Resume. The update with the repaired locator and execution resumes with the recovered element. The application under test is not modified at any point in this flow.

Step 8: Dashboard Update. Every outcome, including refusals, is published to the dashboard together with its score breakdown, reason, and recommendation.

3.3.2 Scoring Algorithm Table

The “Self-Healing” capability is powered by a weighted scoring system. When an element is not found, the framework scrapes the Live DOM and compares all elements against the “Golden Fingerprint” stored in the database.

Table 3.1: Heuristic Scoring Algorithm Table

Rule id	Attribute	weighting	logic
R1	INNER TEXT	40%	Matches the visible label or button text.
R2	XPATH PATTERN	30%	Analyses the tree structure similarity.

R3	CSS CLASS	20%	Matches styling and design attributes.
R4	NEIGHBORS	10%	Checks if surrounding elements are identical.

Note on score computation. The four rules are combined as a weighted sum, $\text{Confidence} = 0.40 \cdot R1 + 0.30 \cdot R2 + 0.20 \cdot R3 + 0.10 \cdot R4$. Where an attribute is absent from both the Golden Fingerprint and the candidate element — for example an input field that carries no visible text — that rule is excluded from the calculation and the remaining weights are normalized, so that the absence of an attribute is not scored as a mismatch. Candidates whose tag family differs from the fingerprint are heavily penalized. An automatic heal additionally requires the winning candidate to exceed the next-best candidate by a minimum margin, so that a page containing two similar elements cannot produce a confident but incorrect substitution.

Note on thresholds. The values of 75 per cent and 20 per cent are the system defaults. Both are configurable at runtime by the administrator through the dashboard, and the configured values are applied to all subsequent healing operations without restarting the framework.

Decision Thresholds:

1. Automatic Healing (Score > 75%): System is highly confident; it fixes the code and continues.
2. Low Confidence (20% – 75%): System attempts a fix but flags the action on the dashboard for review.
3. Manual Intervention (Score < 20%): System determines the UI change is too drastic and stops the test to prevent false clicks.

3.4 System Operational Workflow

The System Operational Workflow describes the chronological execution behavior of the Rule Based Self-Healing Framework. Unlike architectural diagrams that describe structural components, this section explains how the system behaves over time from initialization to recovery.

The system operates in two primary modes:

1. Learning Mode
2. Healing Mode

The transition between these modes is determined by the presence of baseline metadata and runtime failure detection.

3.4.1 Operational Lifecycle

The behavior of the framework is best understood as a sequence of events across two working days of an ordinary development cycle. This lifecycle defines the contract between the developer, the quality assurance engineer, and the framework.

Day 1: Baseline Establishment

1. The developer publishes a working version of the web page.
2. The quality assurance engineer writes a Selenium test script against that page using ordinary locators, with no knowledge of the self-healing framework.
3. The test script executes and passes.
4. Because the run passed, the framework enters Learning Mode and records a Golden Fingerprint for each element the

Day 2: Failure and Recovery

1. The developer modifies the markup of the page, for example by renaming an identifier, changing a CSS class, altering a name attribute, or restructuring the DOM.
2. The quality assurance engineer runs exactly the same test script, unmodified.

3. Selenium fails to resolve the affected locator and raises a `NoSuchElementException`.
4. The framework intercepts the failure before it reaches the test, extracts the live DOM, and scores every candidate element against the stored fingerprint.
5. The rule engine selects the best replacement element, derives the repaired locator from it, and returns the element to the running test script, which continues executing.
6. The dashboard reports the original locator, the repaired locator, the heuristic score with its rule breakdown, the reason for the repair, the confidence level, the log history, and the recommended change for the quality assurance team.

At no point in this lifecycle is the application under test modified. The test script is not required to change between Day 1 and Day 2, and the recovery is applied to its execution rather than to its source.

3.4.2 Learning Mode Workflow

Learning Mode is the phase in which the system records the reference state of the application user interface before any failure occurs. It is activated when a test run completes successfully, either because no baseline exists for the application yet or because the administrator has requested that the baseline be refreshed against a newly approved build.

The defining principle of Learning Mode is that a baseline is recorded only from a verified-correct state. A fingerprint captured from a failing or already-modified page would record the fault itself as the reference, and every subsequent comparison would be measured against a broken model. For this reason, Learning Mode is bound to a passing run, and healing is suppressed for the duration of the capture so that the discovery process cannot generate matches of its own.

Capturing the baseline from the locators the script actually used, rather than from an indiscriminate scan of the page, has two consequences. It restricts the baseline to elements the test suite genuinely depends upon, and it establishes a direct link between a test script and the elements it requires, which is what allows a later failure to be resolved to the element the script intended rather than to the most superficially similar element on the page.

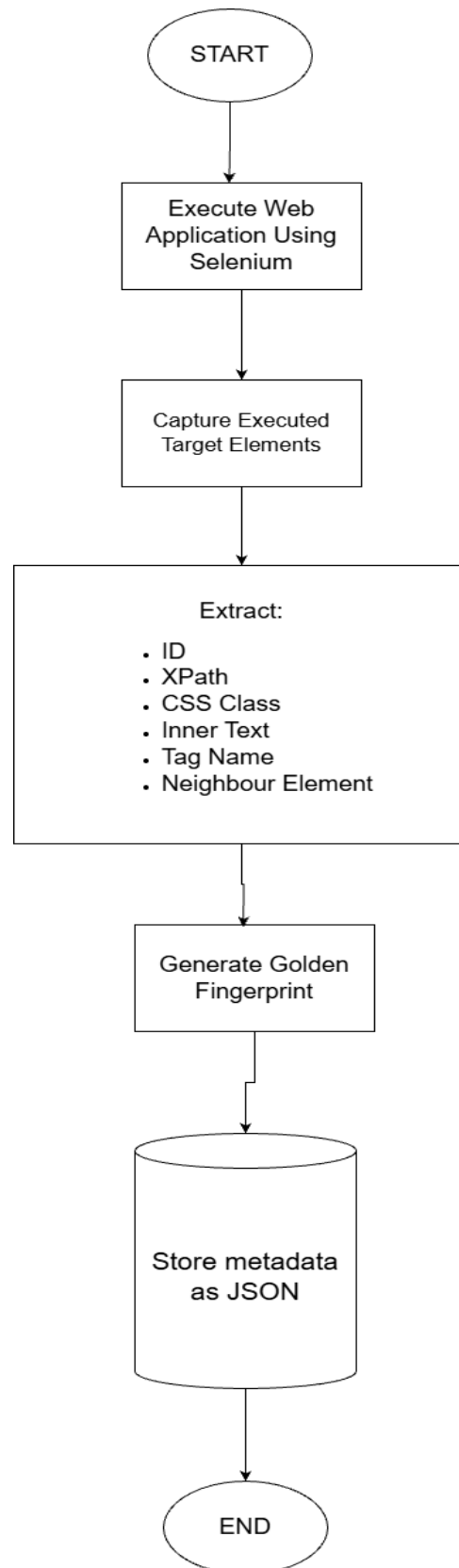


Figure 3.4: Learning Mode Workflow

3.4.3 Healing Mode Workflow

Healing Mode is activated when Selenium fails to locate a previously stored element during runtime execution.

This indicates a potential UI change such as:

1. Modified element ID
2. Updated XPath
3. Structural DOM shift
4. Dynamic attribute variation

The Healing Workflow proceeds as follows:

Step 1: Failure Detection. The framework intercepts the `NoSuchElementException` raised by Selenium before it propagates to the test script, so that the run is suspended rather than terminated.

Step 2: Live DOM Extraction. The current DOM is extracted from the browser in a single operation, and the structural and textual features of every element are collected.

Step 3: Candidate Identification. The failing locator is resolved to the Golden Fingerprint it was recorded against during Learning Mode, establishing which element the script intended to reach. All live elements of a compatible type are collected as candidates for that specific element.

Step 4: Heuristic Similarity Evaluation. Each candidate is compared against the Golden Fingerprint using the weighted rules of Table 3.1:

$$\text{Confidence Score} = (\text{Weight}_1 \times \text{Text Similarity}) + (\text{Weight}_2 \times \text{XPath Structural Similarity}) + (\text{Weight}_3 \times \text{CSS Match}) + (\text{Weight}_4 \times \text{Neighbour Similarity})$$

Each rule contributes proportionally to the final confidence score, and the score is accompanied by its per-rule breakdown so that the decision can be explained.

Step 5: Threshold-Based Decision.

If $\text{Confidence} \geq 75$ per cent, the system performs automatic healing.

If $20 \text{ per cent} \leq \text{Confidence} < 75$ per cent, the system performs cautious healing and flags the event for dashboard review.

If $\text{Confidence} < 20$ per cent, the system halts execution and triggers a manual intervention alert rather than interacting with an element it cannot confidently identify.

Step 6: Repaired Locator Derivation. For an accepted match, the framework extracts the identifying attributes of the winning live element — identifier, name, test-specific data attributes, accessibility label, input type, and link text — and ranks them by expected

stability. The highest-ranked attribute is expressed as the repaired locator and recorded as the replacement for the original.

Step 7: Locator Metadata Update. The locator metadata repository is updated so that the mapping between the test script's logical element and its current locator reflects the repaired value. This update is confined to the framework's own repository. The markup, stylesheets, scripts and source code of the application under test are not modified.

Step 8: Execution Resume. The recovered element is returned to the running test script, which continues from the point of failure. The remainder of the suite executes without restarting, and the corrected locator is reported to the quality assurance team as a recommended change to the test script.

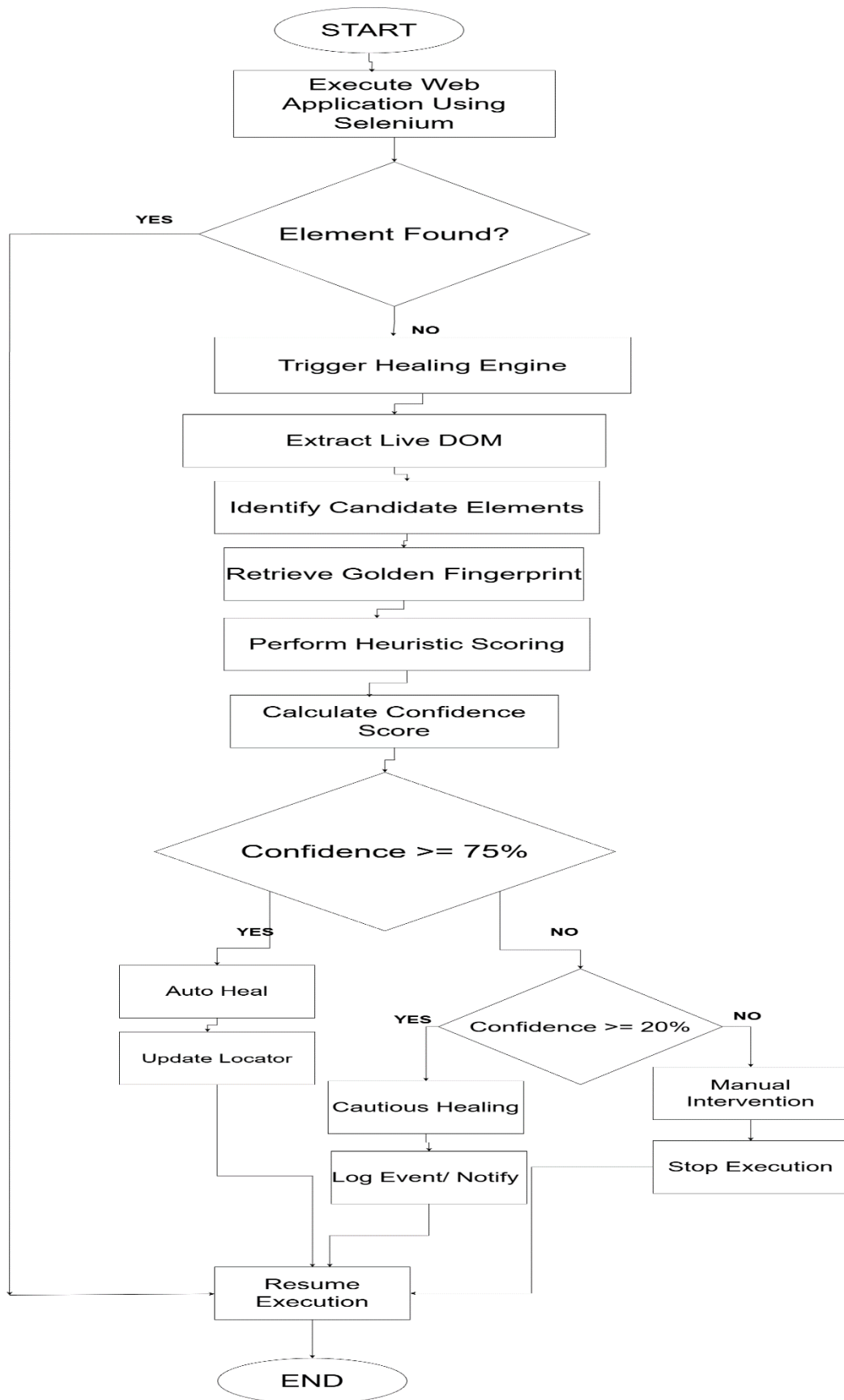


Figure 3.5: Healing Mode Flowchart

3.4.4 Feedback and Update Cycle

After healing action execution, the system performs verification to ensure that the recovery was successful.

The feedback mechanism includes:

1. Post:healing validation of element interaction.
2. Confirmation that the intended UI action was executed correctly.
3. Logging of:
 - a. Healing timestamp
 - b. Original locator
 - c. Updated locator
 - d. Confidence score
 - e. Final status (Success/Failure)
4. Updating the real:time dashboard with healing statistics.
5. Monitoring for repeated failure attempts to prevent infinite recovery loops.

If multiple healing attempts fail consecutively, the system escalates the issue and prevents further automated intervention.

This closed:loop mechanism ensures that recovery actions are verified before execution continues. It transforms the system from a simple reaction engine into a controlled, feedback:based reliability framework.

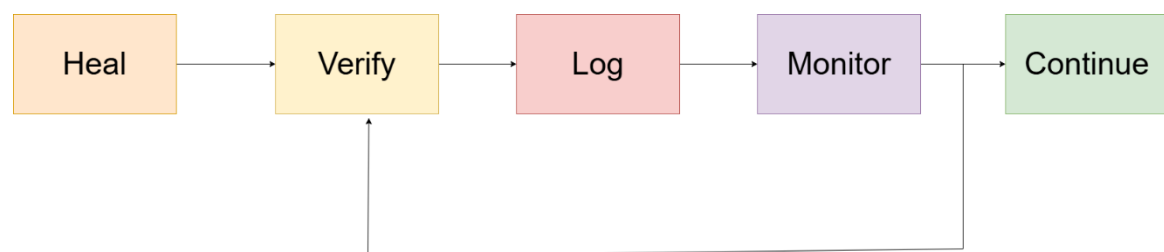


Figure 3.6: Close looped feedback and update cycle

3.4.5 Integration with an Existing Test Suite

The healing layer is designed to be attached to a test suite that knows nothing about it. The suite used for evaluation is an ordinary Selenium page:object suite with explicit waits and behavioural assertions, and it contains no reference to the framework of any kind. This is deliberate: keeping the suite unaware of the layer preserves it as an experimental control, so that the only variable between a baseline run and a healed run is whether the layer is installed.

Three integration levels are provided, all sharing one healing core:

- Transparent proxy. The driver is wrapped once at the point of creation. Every call other than an element lookup is delegated untouched to the real driver, so the wrapper is substitutable for one and page objects, waits and script execution continue to work unchanged.
- In:place patch. A single install call patches the element:lookup methods on both the driver and the element classes, so lookups nested inside page objects are covered as well. Because the expected conditions used by explicit waits call those same methods internally, a wait that would previously have timed out is instead satisfied by a heal on its first attempt.
- Test:runner plugin. The layer is enabled by a command:line flag on the test runner, so the suite itself requires no change whatsoever.

Two safeguards prevent the framework from manufacturing a recovery where no failure occurred. First, baseline learning runs with healing suppressed, so that the discovery scan Learning Mode uses to build a baseline cannot be healed by the very engine that baseline is being built for. Second, healing of plural element lookups is opt:in rather than enabled by default, because an empty result list is a legitimate answer to such a lookup the correct response to “are there any error banners on screen?” is frequently none whereas a raised NoSuchElementException is an unambiguous failure signal. With both safeguards in place, a clean run produces exactly the heals that the injected faults require and raises no spurious alerts.

3.5 Software Development Life Cycle

Development Methodology: Incremental Development Model

The development of the Self:Healing Framework followed the Incremental Development Model. This model was selected because the system could be logically divided into independent, functional increments, each delivering a usable component of the framework. Instead of waiting for the entire system to be complete, core capabilities such as data capture, scoring, healing logic, and visualization were developed and validated incrementally.

This approach was particularly suitable for the Scoring Engine, as rule accuracy and confidence thresholds needed validation against real:world website changes before adding higher:level features like dashboards and reporting.

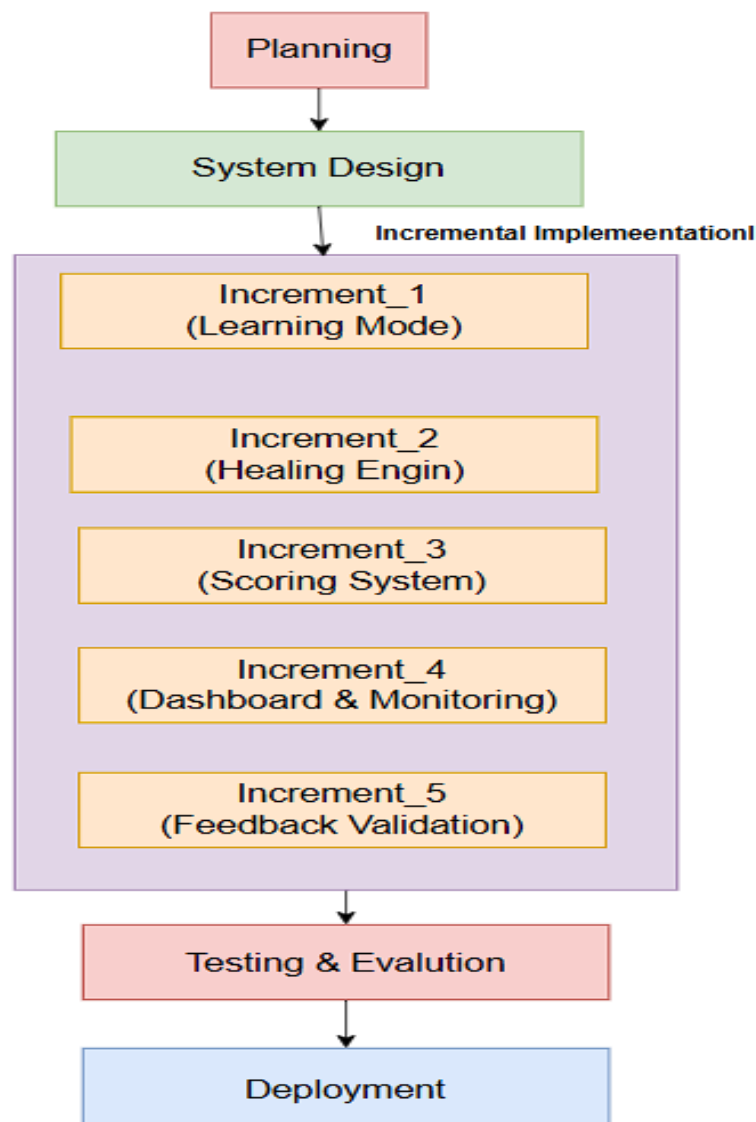


Figure 3.7: Incremental SDLC Model of Self:Healing System

3.5.1 Phases of Development

3.5.1.1 Planning and Requirements Analysis

In this phase, common automation failure points were identified through analysis of real test failures. These included:

1. Dynamic element IDs
2. Changing XPath and CSS selectors
3. DOM structure modifications

Based on these observations, a confidence: based healing mechanism was defined. A 75% confidence threshold was established to determine whether the system should auto: heal or request manual intervention.

3.5.1.2 System Design

A layered architecture was designed to support incremental growth and isolation of responsibilities. The five architectural layers described in Section 3.1.2 are realized in implementation as four functional groupings:

- Execution Layer: Runs the automation scripts and intercepts locator failures (corresponds to the Request and Application Layer and to the Healing and Execution Layer).
- Core Engine Layer: Applies the heuristic rules, computes confidence scores, and makes threshold decisions (corresponds to the Decision Layer).
- Database Layer: Stores element fingerprints, locator metadata, and historical healing data (supports all layers).
- Dashboard Layer: Visualizes healing decisions and system behavior (corresponds to the Visibility and Control Layer).

3.5.1.3 Incremental Implementation

The system was developed in multiple increments, with each increment delivering a complete and testable feature set.

Increment 1: Learning Mode Module

1. Implemented the Learning Mode to capture UI element fingerprints
2. Stored attributes such as tag name, text, XPath, CSS selectors, and surrounding context in JSON format
3. Provided the foundation for all subsequent healing logic

Increment 2: Heuristic Scoring Engine

1. Developed the Scoring Engine using predefined heuristic rules (R1–R4)

2. Assigned weighted scores to element attributes to calculate a confidence percentage
3. Enabled decision: making based on score thresholds

Increment 3: Monitoring and Visualization Dashboard

1. Integrated a Streamlit based dashboard
2. Displayed real: time healing attempts, confidence scores, and fallback actions
3. Improved system transparency and debugging capability

Increment 4: Suite Integration and Fault: Injection Harness

- Added the three integration levels described in Section 3.4.5, so the layer can be attached to an existing Selenium suite without editing its test code
- Built a repeatable fault: injection harness that refactors the target application and runs the same unmodified suite with and without the layer installed, producing a directly comparable measurement
- Extended the framework to server: side infrastructure recovery and browser: side anomaly detection, feeding one shared log and one dashboard

3.5.2 Why Incremental Model Fits Better

1. Each module (Learning, Scoring, Dashboard) delivers independent value
2. Early increments were usable even before full system completion
3. Risk was reduced by validating logic increment by increment
4. Rule tuning occurred *within* increments, not as endless rework of the same build

In short, our project adds capabilities step:by:step, not just refinement cycles making the Incremental Development Model the correct and defensible choice for both academic evaluation and technical justification.

This framing will stand strong in front of judges and examiners because it aligns methodology with actual system behavior, not buzzwords.

3.6 Technology Stack

The tools were selected for determinism, absence of external service dependencies, and suitability for a controlled single-server environment.

1. Programming Language: Python 3.10 or later.
2. Browser Automation: Selenium WebDriver 4.x, with WebDriver-manager for driver resolution. Chrome and Firefox are both supported.
3. DOM Extraction: injected JavaScript executed through the WebDriver session. The structural and textual features of all candidate elements are collected in a single

round trip, which keeps the cost of a heal to a few tens of milliseconds. A server-side HTML parsing library is deliberately not used, because the framework must observe the rendered DOM as the browser has constructed it, including any modifications made by client-side scripts, rather than the static markup returned by the server.

4. Metadata and Telemetry Storage: JSON documents written atomically, with cross-process file locking provided by file lock, so that concurrently executing test processes cannot corrupt shared state.
5. Dashboard Framework: Streamlit.

3.7 The Real: Time Dashboard

A monitoring and reporting interface is a required component of the methodology, because an autonomous recovery mechanism is only acceptable to a quality assurance team if every decision it makes is visible and explainable. The dashboard provides the following:

1. Original Locator: The locator recorded in the test script that failed to resolve, together with the test name and script location that requested it.
2. Repaired Locator: The locator derived from the matched live element, expressed as a strategy and value that the quality assurance engineer can apply directly to the test script.
3. Heuristic Score: The composite confidence value together with its per-rule breakdown across R1 (text), R2 (XPath), R3 (CSS class) and R4 (neighbor's), so that the basis of the decision is visible.
4. Reason for Repair: A generated explanation stating which attributes matched, which changed, and which rules determined the outcome, for example that the visible text and DOM position were unchanged while the CSS class had been renamed.
5. Confidence and Decision: The band the score fell into and the resulting action: automatic heal, cautious heal flagged for review, or refusal.
6. Log History: The chronological record of all healing events, grouped by test run, together with healing rate and success rate over time.
7. Recommendations for Quality Assurance: The specific change suggested for each affected test script, and aggregate guidance identifying locators that heal repeatedly and are therefore candidates for refactoring.

8. Baseline View: The Golden Fingerprints recorded, and from which build, so that the Day 1 reference state can be inspected and compared against the current page.
9. Alerts: Notification when a confidence threshold and manual intervention is required, or when a single locator has failed to heal repeatedly and automatic intervention has been suspended.
10. Configuration: Administrative control of the confidence thresholds and healing behavior without editing source code.

3.8 Testing & Evaluation

The framework is evaluated using a controlled two-phase protocol that reproduces the operational lifecycle described in Section 3.4.1.

Phase 1: Baseline. A test suite is executed against a known-good build of the application under test. The run is verified to pass in full, and the resulting Golden Fingerprints are recorded as the reference state.

Phase 2: Mutation and Recovery. A defined set of markup mutations is applied to the application, each representing a change a developer might make during ordinary development:

1. Element identifier renamed
2. CSS class renamed
3. name or data attribute changed
4. Visible text altered
5. DOM structure modified by insertion or relocation of a container element
6. Multiple attributes changed simultaneously
7. Element removed entirely, as a negative control

The identical, unmodified test suite is then executed against the mutated build, first without the framework installed to establish the baseline failure rate, and then with the framework installed. Because the test scripts, the page objects, and the application are identical between the two executions, the only variable is the presence of the healing layer.

The following measures are recorded:

1. Healing success rate: The proportion of locator failures from which execution was successfully recovered.
2. Correct-identification rate: The proportion of heals that resolved to the intended element, verified against the known mutation, since a heal to the wrong element is a failure even when the test passes.
3. Correct-refusal rate: The proportion of cases in the negative control, where the element genuinely no longer exists, in which the system correctly halted rather than substituting an incorrect element.
4. Confidence distribution: The distribution of scores relative to the 75 per cent and 20 per cent thresholds.
5. Heal latency: The time taken to recover from a single locator failure.

6. Baseline recovery: Verification that the fingerprint repository is rebuilt correctly after deliberate deletion.

The correct-refusal measure is reported alongside the success rate because a system that heals everything is not a safe system; the ability to decline is treated as a functional requirement rather than a limitation.

3.9 Hardware and Software Requirements

1. 1 Hardware: Minimum 8GB RAM, i5 Processor (for concurrent DOM processing).
2. 2 Software: Windows 10/11, Python Environment, Chrome/Edge Web drivers.

CHAPTER 4: RESULT AND ANALYSIS

4.1 Evaluation Setup

This chapter reports the outcomes measured after the framework was implemented, together with an interpretation of what those outcomes mean for the objectives set out in Section 1.3. The evaluation was not carried out on paper or against previously recorded logs: every figure reported below was produced by re:running the scripted harness described in Section 3.8 against a live stack the target application served locally, a real Google Chrome instance driven by Selenium WebDriver in headless mode, and the dashboard receiving telemetry as the runs proceeded. Because the harness is fully scripted, each result is reproducible with a single command, and the headline figures were confirmed stable across repeated runs.

The evaluation is built around one deliberate experimental control. The test suite used throughout is an ordinary Selenium page:object suite: it uses explicit waits, asserts on user:visible behaviour, and contains no reference of any kind to the healing framework. The test files, the page objects and the application build are identical between a baseline run and a healed run. The only variable is whether the healing layer is installed. This distinction matters, because it makes the comparison evidence rather than illustration any difference in outcome between the two runs is attributable to the framework and to nothing else.

The fault injected for the headline experiment reproduces the way locator rot actually occurs in practice. Rather than corrupting the page in a way that would visibly break it, the application is served in a refactored build in which element identifiers, CSS class names and a data attribute are renamed consistently: `start:btn` becomes `btn:start:primary`, `reset:btn` becomes `btn:reset:secondary`, `skip:btn` becomes `btn:skip:forward`, and the shared `btn:main` class becomes `btn:cta`. Because the renaming is consistent, the application continues to work perfectly for a human user the timer counts down, every button responds, and nothing on screen is wrong. Only the locators recorded by the test suite have gone stale. This is precisely the change that is correct from the developer's point of view and catastrophic from the test suite's, and it is the scenario the framework is designed to absorb.

All measurements reported in this chapter were taken on Windows 11 with Python 3.13 and Google Chrome running in headless mode.

4.2 Locator: Healing Results

4.2.1 Baseline versus Self: Healed Execution

The headline experiment runs the same five: test suite twice against the same refactored build: once on stock Selenium, and once with the healing layer installed through a single line of integration code. Table 4.1 reports the two runs side by side.

Table 4.1: Baseline versus Self: Healed Execution of the Same Test Suite

Measure	Baseline (stock Selenium)	With self: healing layer
Tests passed	1 of 5	5 of 5
Tests recovered	Not applicable	4
Locator heals performed	0 (no healing available)	5
Mean time per heal	Not applicable	29.1 ms
Suite wall: clock time	16.6 s	2.4 s
Failure modes observed	1 NoSuchElementException, 3 TimeoutException	None
Lines of test code changed	0	0
Lines of integration code added	0	1

Four of the five tests failed under stock Selenium, and all four were recovered with the layer installed, taking the suite from 1/5 to 5/5 without a single edit to the test code. The fifth test passed in both runs and triggered no heal. That is a useful negative result in its own right: the framework did not intervene where there was nothing to repair, which is the behavior required if a healing layer is to be trusted in a suite where most tests are healthy. The second observation is less obvious and, in practice, at least as valuable. The failing baseline run took 16.6 seconds while the healed run took 2.4 seconds the broken suite was roughly seven times slower than the working one. This is not because healing is fast; it is because a broken locator is expensive. Only the first failure surfaced immediately as a `NoSuchElementException`, at 0.09 seconds. The remaining three surfaced as `TimeoutException` only after each explicit wait had burned its full five:second timeout, at 5.11, 5.11 and 5.13 seconds respectively. Locator rot therefore imposes two costs on a team rather than one: red builds and slow builds. Because the framework resolves the lookup inside `find_element` itself, the wait condition is satisfied on the first attempt and never times

out, so both costs are removed together. At a mean of 29.1 milliseconds, a heal is roughly two orders of magnitude cheaper than the wait timeout it prevents.

The heal cost was stable across repeated runs, measured at 28.4 and 29.1 milliseconds in two consecutive executions. This indicates that the single:pass document extraction described in Section 3.6 keeps the recovery path cheap enough for the layer to be left installed permanently, rather than enabled only when a suite is already known to be failing.

4.2.2 Confidence Scoring and Rule Contributions

Every heal is recorded together with the confidence score that authorised it and the individual R1–R4 component scores that produced that score, so no healing decision is opaque. The healed run performed five heals against three distinct elements; Table 4.2 reports the distinct cases.

Table 4.2: Confidence Scores and Rule Contributions for Each Healed Element

Broken locator	Element recovered	R1 / R2 / R3 / R4 (%)	Confidence and decision
id='start:btn'	<button> “START”	100 / 100 / 78.26 / 100	95.65% AUTOMATIC HEAL
id='reset:btn'	<button> “Reset”	100 / 100 / 0 / 84.78	78.48% AUTOMATIC HEAL
id='skip:btn'	<button> “Skip”	100 / 100 / 0 / 100	80.00% AUTOMATIC HEAL

The pattern across these three rows is the clearest evidence in the project that the weighting scheme of Table 3.1 is doing real work rather than decorating a decision that had already been made. All three elements were re:identified from intact visible text and an unchanged position in the document tree, so R1 and R2 contributed their full 40% and 30% in every case. The rows differ only in the two lower:weighted rules, and they differ for reasons that are visible in the application’s own markup.

The START button carries a class attribute that the refactor renamed, so R3 returned a partial 78.26% rather than a failure, and the element still scored 95.65%. The Reset and Skip buttons carry no class attribute at all, so R3 contributed exactly zero for both not because the match was poor, but because there was nothing for that rule to compare. Both

nevertheless cleared the 75% automatic gate, at 78.48% and 80.00%, because the 20% that R3 can contribute is not decisive on its own. Reset scored marginally lower than Skip because its neighbouring elements had shifted slightly, reducing R4 to 84.78% against Skip’s 100%.

This is the intended consequence of distributing weight across four independent attributes rather than trusting any single one. A scheme that depended primarily on the CSS class would have failed outright on Reset and Skip; a scheme that depended on the element identifier would have failed on all three, since the identifier is exactly what the refactor changed. Spreading the evidence means that the loss of any one attribute degrades confidence proportionally instead of destroying the match.

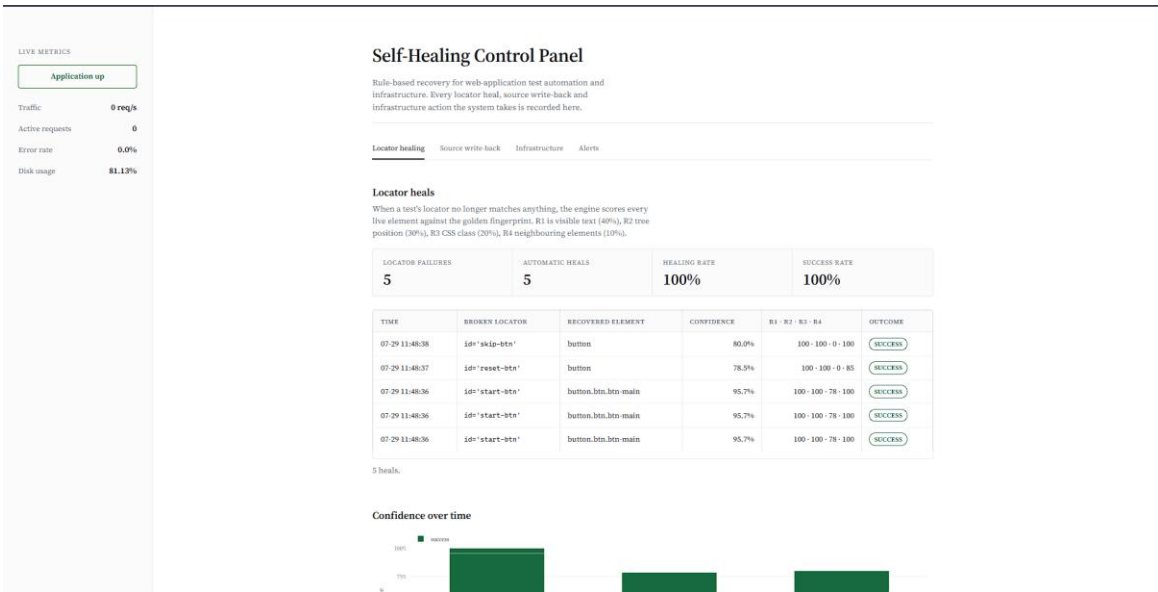


Figure 4.1: Dashboard UI Heuristic Healing tab, showing each heal with its confidence score and R1–R4 breakdown

4.2.3 Isolated Front:End Change Scenarios

The headline experiment changes several attributes at once, which is realistic but makes it harder to attribute a recovery to any particular kind of change. A second, narrower harness therefore applies four faults in isolation, each mutating the same START button in one specific way that a developer might, and then requests the now:stale locator. Table 4.3 reports the outcome of each.

Table 4.3: Recovery Outcomes Across Isolated Front:End Change Scenarios

Scenario	App:side change	Stale locator requested	Observed outcome
----------	-----------------	-------------------------	------------------

CSS class renamed	btn:main → btn:primary	css selector = 'btn:main'	Recovered by the in:browser agent, which restored the fingerprinted class in the live DOM before the lookup ran; no heuristic heal was required
Element id renamed	start:btn → start:btn:v2	id = 'start:btn'	Recovered by the heuristic engine at 100% confidence and logged as an automatic heal
Data attribute changed	data:action “start” → “begin”	css selector = '[data:action=“start”]'	Recovered by the heuristic engine at 100% confidence and logged as an automatic heal
Multiple changes at once	class, id and data attribute all changed	css selector = 'btn:main'	Recovered by the in:browser agent, as in the first scenario; the combined change did not defeat recovery

All four scenarios were recovered, but they were not all recovered by the same engine, and the distinction is worth stating precisely because it shows the framework operating in layers. The two scenarios in which the element identifier or the data attribute was renamed were recovered by the heuristic engine: the stale locator raised `NoSuchElementException`, 34 live candidates were extracted and scored, and the START button was re:identified at 100% confidence and logged to the UI Heuristic Healing tab.

The two scenarios in which the CSS class was renamed were recovered earlier in the chain, by the in:browser agent. That agent watches a configured selector and, when it cannot find

it, re:identifies the target from a stored text fingerprint and restores the fingerprinted class attribute onto the live element. By the time Selenium performed its lookup the class had already been restored in the live document, so the stale locator resolved directly and the heuristic engine was correctly never invoked. Both routes produce a passing test, but only the heuristic route produces a dashboard row which is why this run records four recoveries and two logged heals rather than four of each.

Read together, these scenarios show that the framework has more than one line of defence against front:end change, and that each engine correctly declines to act when an earlier one has already resolved the fault.

4.2.4 Permanent Source:Level Self:Correction

Runtime recovery keeps a suite running, but on its own it does not reduce the maintenance backlog: the stale locator is still stale the next time the suite runs. The framework therefore writes a high:confidence heal back into the automation source. Exercised against a script whose locator had been deliberately set to a non:existent identifier, the framework recovered the element at 100% confidence and then rewrote the line on disk, changing the recorded identifier from old:start:btn to start:btn. The write:back was recorded as a distinct event one occurrence patched in the target file and a backup copy of the original file was retained, so the edit is both auditable and reversible.

Re:running the same script afterwards produced no heal at all, because the locator on disk was now correct and the element resolved directly. This confirms that the correction is permanent rather than per:run: the framework does not merely absorb the same failure repeatedly, it removes it.

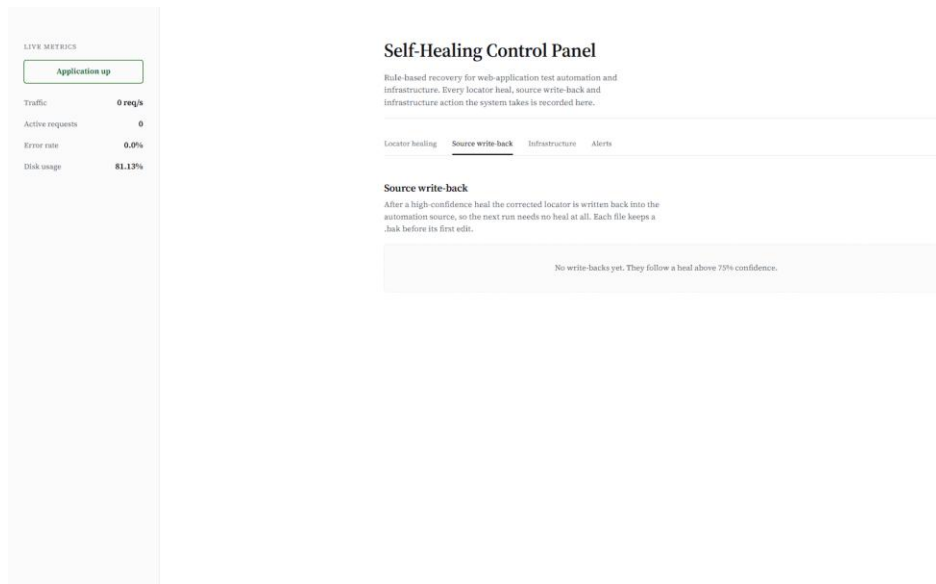


Figure 4.2: Dashboard Locator Self:Correction tab, showing the healed locator written back to the automation source

4.2.5 Intent: Aware Healing

A heuristic engine that always returned its single best:scoring candidate would appear to work on a page with one obvious target and would fail quietly on a page with several. To test this, the tracked locator was repointed at a different element the Focus mode button rather than START and broken in the same way. The framework recovered the Focus button, not START. The broken locator is first matched to the fingerprint it was intended to identify, and scoring then proceeds against that fingerprint, so two different broken locators on the same page heal to two different elements rather than both collapsing onto the highest:scoring candidate on the page.

4.2.6 Baseline Recovery After Fingerprint Loss

The framework depends on a stored baseline, which makes the loss of that baseline a legitimate failure mode to test rather than an inconvenience to avoid. With the fingerprint metadata file deleted, the next healing run detected its absence, entered Learning Mode automatically, rescanned the live application and rebuilt the Golden Fingerprint baseline from the interactive elements it found, and then completed the heal that had triggered it. No manual reconstruction was required. Baseline learning runs with healing suppressed, so the discovery scan cannot itself be healed by the engine whose baseline it is building.

4.3 Safety Behaviour: Manual: Intervention Halt

A framework that attempts a fix regardless of confidence is arguably worse than no automation at all, because a test that silently interacts with the wrong element reports a false pass the most expensive outcome a test suite can produce. The low:confidence boundary was therefore tested explicitly, in two ways.

In the first, a locator was set to reference an element with no reasonable counterpart on the page. The best available candidate scored 12.0%, below the 20% safety gate. The framework declined to act: it raised a critical entry under the Alert Notification Logs tab recording both the score and the refusal, made no source:code edit, and re:raised the original exception to the calling test so that the failure remained visible rather than being absorbed. In the second, a locator was pointed at a non:interactive display element whose best candidate scored 0.0%, with the same result.

The distinction between these outcomes and an ordinary heal is the important one. Between 20% and 75% the framework reroutes the test but marks the row for review, so execution continues under supervision; below 20% it refuses outright and surrenders control to a human. The framework therefore fails safe rather than failing silent, and the confidence score is not merely reported it is the mechanism that decides whether the framework is permitted to act at all.

A related safeguard was verified separately. When the same locator fails to heal three times in succession, the framework locks that locator, escalates it as a critical alert and stops attempting further automated recovery, which prevents a test from entering an unbounded heal:and:retry loop.

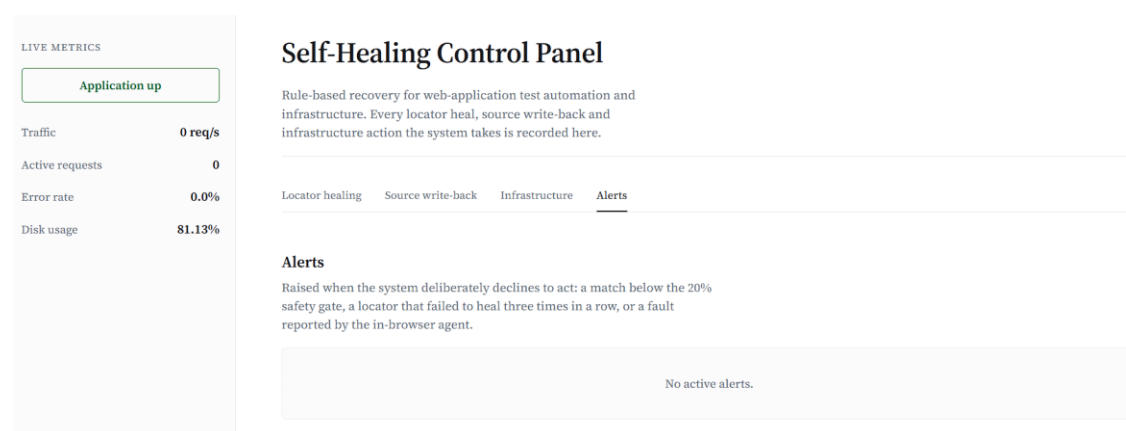


Figure 4.3: Dashboard Alert Notification Logs tab, showing a safety:gate refusal recorded as a critical entry

4.4 Infrastructure: Healing Results

The same detect:score:act pattern was exercised against server: side faults, by two methods. In the first, a simulation script drives the monitored metrics into a fault state directly. With disk utilization reported at 86.8%, above the configured 85% threshold, the rule engine selected and executed its log: rotation and temporary: file purge action and recorded the outcome as resolved under the Server Infrastructure Heals tab.

The second method is the more convincing of the two, because nothing about it is simulated. Repeated real requests were issued against an endpoint that returns HTTP 500 under load. The background monitor, which polls the running application on a fixed interval and was given no instruction to expect anything, observed the resulting pattern HTTP status 500 with an error rate of 100% and executed a graceful worker:pool reload within seconds, unprompted. A further run under combined stress (status 500, error rate 73.0%, disk utilisation 92.4%) triggered both the recovery action and a critical disk:space alert, confirming that recovery and escalation are independent responses that can fire together rather than alternatives to one another.

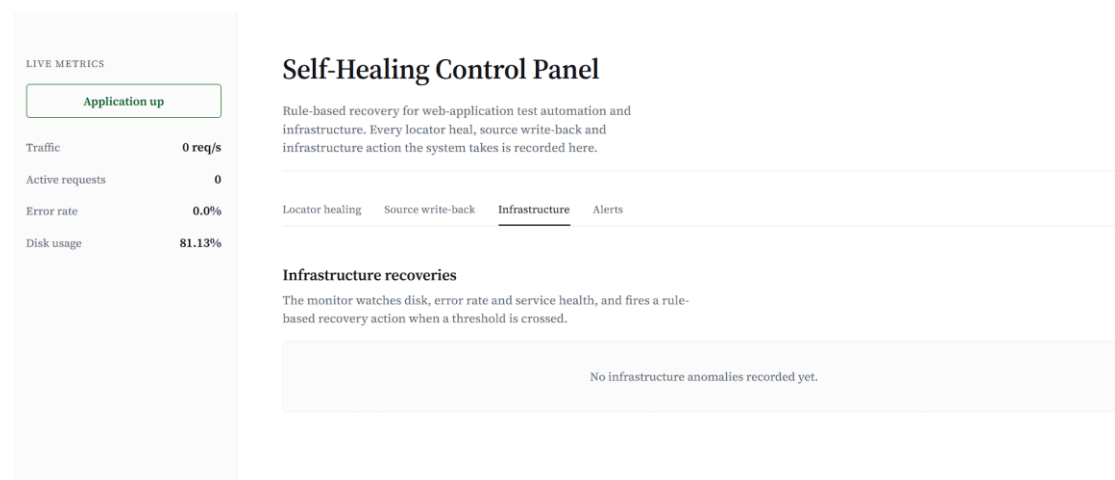


Figure 4.4: Dashboard Server Infrastructure Heals tab, showing a rule:based recovery action and its trigger metrics

4.5 Browser:Side Anomaly Detection

The third engine addresses a category of fault that neither of the other two can observe. The target page was loaded normally in a real browser, rather than through the automated suite, with a resource on the page deliberately unavailable. Because the server returns its response correctly and the failure occurs only after that response reaches the client, server:side monitoring records nothing unusual. The in:browser collector captured the failure and

routed it into the shared alerting pipeline, where it appeared under Alert Notification Logs as a distinct browser: side anomaly. A JavaScript runtime error raised on the same page was captured through the same route and logged as a warning.

The result confirms the framework’s coverage claim in the specific sense that matters: it does not merely add a third monitor, it makes visible a class of failure that would otherwise have gone unreported entirely.

4.6 Automated Verification Harness

Alongside the scenario:level demonstrations, an automated harness asserts the framework’s decision logic directly, so that a regression in the rules can be detected without a browser or a full test run. The harness drives the engine through each confidence band and then inspects the telemetry that resulted. Table 4.4 reports the outcome.

Table 4.4: Automated Verification Harness Results

Check	Property verified	Result
High:confidence decision	A score of 100% yields a continue decision	Pass
High:confidence recovery	The correct element is returned to the calling test	Pass
Mid:confidence decision	A score of 55% yields a cautious verify decision	Pass
Low:confidence decision	A score of 12% yields a halt decision	Pass
Healing log contents	Both successful and flagged heals are recorded	Pass
Alert routing	A below:threshold refusal is written to alerts and not to the healing log	Pass
Infrastructure log	A recovery action is recorded together with its trigger metrics	Pass
Disk escalation	Disk utilisation above 90% raises a critical alert	Pass

Browser telemetry	Client:side events are ingested through the passive path	Pass
-------------------	--	------

All nine checks passed. The harness confirms not only that each confidence band produces the correct lifecycle decision continue, verify or halt but also that outcomes are routed to the correct destination: a successful heal and a flagged cautious heal both appear in the healing log, whereas a below:threshold refusal is written to the alerts stream and deliberately not recorded as a heal, so that the healing statistics are never inflated by actions the framework in fact declined to take.

4.7 Consolidated Results

Table 4.5 draws the individual results together across all three fault domains, recording for each case the trigger that was applied and the outcome that was observed.

Table 4.5: Demonstrated Outcomes Across All Three Fault Domains

Domain	Case	Trigger applied	Observed outcome
Locator	Suite:level recovery	Consistent front:end refactor of identifiers, classes and a data attribute	Suite moved from 1/5 to 5/5 with no test:code change; five heals at a mean of 29.1 ms
Locator	Automatic heal	Known element with a renamed identifier	Recovered at 95.65% and logged with its full R1–R4 breakdown
Locator	Reduced:evidence heal	Elements carrying no class attribute	Recovered at 78.48% and 80.00%; R3 contributed zero without defeating the match
Locator	Source self:correction	The same failure run against the actual script file	Source line rewritten on disk, backup retained,

			and no heal needed on re:run
Locator	Intent:aware heal	Locator repointed at a different tracked element	The intended element was recovered rather than the highest:scoring candidate
Locator	Baseline loss	Fingerprint metadata file deleted	Learning Mode rebuilt the baseline automatically and then completed the heal
Locator	Safety halt	Locator with no reasonable match on the page	Refused at 12.0% and 0.0%; critical alert raised, no source edit, exception re:raised
Locator	Repeated failure	Three consecutive failed heals on one locator	Locator locked and escalated; no further automated attempts made
Infrastructure	Simulated fault	Disk utilisation driven to 86.8%	Log:rotation and purge action executed and recorded as resolved
Infrastructure	Real:load fault	Repeated requests to an endpoint	Monitor detected status 500 at a 100% error rate and

		returning HTTP 500	reloaded the worker pool unprompted
Browser	Client:only anomaly	Page loaded in a real browser with an unavailable resource	Anomaly captured and logged although invisible to server:side monitoring

4.8 Analysis

Across every case exercised, the framework’s observed behaviour matched the decision logic specified in Chapter 3, and did so at both ends of the confidence range. High:confidence matches were healed automatically and transparently; an ambiguous match was healed but flagged for review; and matches below the safety gate were refused outright. The value of the design is not that it recovers a broken locator – several commercial tools do that – but that the same score which authorises a recovery also constrains it, so that the framework’s willingness to act degrades in proportion to the evidence available to it.

The most instructive pattern in the results is the relationship between the amount of surviving evidence and the resulting confidence. The three healed elements in Table 4.2 differ only in how much of their fingerprint remained intact, and their scores track that difference closely: 95.65% where the class attribute survived in modified form, and 80.00% and 78.48% where the elements never carried a class at all. No case in the evaluation produced a score in the ambiguous 20–75% band by accident; that band was reached only when the harness deliberately constructed a partial match. This suggests that the weights of Table 3.1 are, at least for elements of this kind, distributed sensibly: a genuine match with one degraded attribute stays comfortably above the automatic gate, while an unrelated element falls far below the safety gate rather than landing between the two.

It is worth noting how widely separated the observed scores are. Genuine matches scored between 78% and 100%, while refusals scored 12.0% and 0.0%; nothing in the evaluation landed near either threshold. A margin of this width is desirable, because it means the thresholds are not finely balanced against the data and a small further change to an element would not flip a decision from automatic recovery to refusal. It is also, however, partly a consequence of the size of the application under test. On a page carrying many visually similar elements, competing candidates would score closer together and that margin would narrow, which is the condition under which the thresholds would need re:examination.

Table 4.6: Fulfilment of the Specific Objectives

Specific objective (Section 1.3.2)	Evidence from this chapter	Status
Capture a baseline Fingerprint of each element's key attributes	Golden Learning Mode captured the interactive elements of the live application and rebuilt the baseline unaided after the metadata file was deleted	Met
Implement a heuristic scoring engine that decides to auto:heal, cautiously heal or halt	All three bands were exercised and produced the specified decision: 95.65% and 100% automatic, 55% cautious, and 12.0% and 0.0% halt	Met
Maintain unified healing logs and a real:time dashboard across all three domains	Locator heals, source write:backs, infrastructure actions and browser anomalies were all recorded through one store and surfaced on four dashboard tabs	Met
Evaluate the framework using fault:injection testing and measure healing accuracy, false:heal rate and reduction in manual intervention	Fault injection was scripted and repeatable; every recovery matched the intended element, no false heal occurred in a clean run, and four failing tests were recovered with zero manual edits. Accuracy is reported over a small scenario set rather than a large statistical sample	Met in part

Interpreted against the problem statement of Section 1.2, the results address the first of the three stated problems directly and the other two by demonstration. Test:suite brittleness was the problem the framework was built to solve and the one measured most rigorously: a refactor that broke four of five tests cost nothing at all once the layer was installed, and the repair was made permanent in the source rather than merely absorbed at runtime. Infrastructure downtime and undetected client:side faults were addressed by showing that the same detect:score:act pattern generalises to them, which supports the project’s central claim that one lightweight framework can cover more than one failure domain without machine learning or per:domain bespoke tooling. The depth of evidence in those two domains is not comparable to that behind the locator engine, and this report does not claim that it is.

One result deserves particular mention because it was not anticipated. In early runs the framework recorded heals that had no corresponding failure most conspicuously a match to a selection element on a page that contained none, logged at 80% confidence. The cause was structural rather than an error in the scoring: the discovery scan that Learning Mode uses to build its baseline probes the page with plural element lookups, and those probes were themselves being healed, so the framework was healing its own learning pass and inventing matches from it. Two changes resolved the problem. Baseline learning now runs with healing suppressed, and the healing of plural lookups became opt:in, on the reasoning that an empty result list is a legitimate answer to a question such as “are there any error banners on screen?”, whereas a raised NoSuchElementException is an unambiguous failure signal. After both changes, a clean run records exactly the heals that the injected faults require and raises no spurious alerts. The episode is reported here rather than omitted because it identifies the sharpest risk in any self:healing layer: a false heal is more dangerous than an unhealed failure, because it converts a visible error into a silent one.

Finally, the scope of these results should be stated plainly. They were obtained against a single application of modest size, using a scripted and therefore finite set of injected faults, and the accuracy figures are counts of scenarios rather than statistics drawn from a large sample. The results establish that the mechanism works and that its safety boundaries hold; they do not establish an accuracy rate that could be generalised to an arbitrary production suite. Section 5.2 identifies the broader evaluation that would be required to do so.

4.9 Comparison with Traditional Selenium Testing

Table 4.7 sets the framework against the conventional approach to the same suite, using the measured figures reported earlier in this chapter wherever a direct measurement exists rather than a qualitative claim.

Table 4.7: Comparison with Traditional Selenium Testing

Aspect	Traditional testing	Selenium Proposed self:healing framework
Broken locator handling	The test fails immediately	The failure is intercepted and the element re:identified from a stored fingerprint
Outcome under a front:end refactor	1 of 5 tests passed	5 of 5 tests passed
Recovery mechanism	Manual correction by an engineer	Automatic, with the decision governed by a confidence threshold
Cost of a broken locator	Up to a full wait timeout per lookup (5 s observed)	A mean of 29.1 ms per heal
Suite execution time under locator rot	16.6 s	2.4 s
Source:code update	Manual	Written back automatically after a high:confidence heal, with a backup retained
Behaviour when the change is too large	Not applicable; the test simply fails	Recovery is refused below 20% confidence and escalated to a human reviewer
Monitoring dashboard	Not available	Available, with the confidence breakdown behind every decision

Infrastructure	and	Not available	Available through the same
browser:side recovery			rule:based pattern and the same unified log
Test:code changes required		Not applicable	None; integration is a single line
Maintenance effort		High	Reduced

The comparison should be read with one qualification. The framework does not make a test suite correct; it makes a correct test suite resilient to a change that was never functional in the first place. Every recovery reported above restored a test to the behaviour it was already asserting, and none of them concealed a genuine defect in the application because a genuine defect changes what the application does, not merely how its elements are labelled. That boundary is precisely what allows automatic recovery to be safe: the framework repairs the reference to an element, never the behaviour being tested.

4.10 Summary of Findings

The results establish that the rule:based self:healing framework recovers reliably from routine front:end change without any edit to the test suite, taking a suite from one passing test to five under a refactor that had broken four of them, at a mean cost of 29.1 milliseconds per recovery and with a sevenfold reduction in wall:clock time. It writes high:confidence repairs back into the automation source, so the same failure is not absorbed twice. It recovers the element that actually failed rather than the highest:scoring candidate on the page, and it rebuilds its own baseline unaided when that baseline is lost. Most importantly for a system that acts autonomously, it refuses to act when the evidence is insufficient, escalating to a human instead of guessing, and it locks a locator that repeatedly fails to heal rather than retrying it indefinitely. The infrastructure and browser:side engines demonstrate that the same rule:based pattern extends beyond the user interface, with results consistent with the design although evaluated less deeply. Locator healing remains the project's flagship and most rigorously measured capability.

CHAPTER 5: CONCLUSION AND FUTURE ENHANCEMENT

5.1 Conclusion

This project set out to address a reliability problem that engineering teams routinely absorb as a cost of doing business. Web applications fail in at least three distinct ways: user:interface test locators go stale as the front end changes, server:side infrastructure degrades under load, and browser:side faults occur that never reach server monitoring at all and each is conventionally handled by a different tool, a different on:call process, or by no automation whatsoever. The observation on which this project rests is that all three can be served by a single pattern: detect a deviation from a known baseline, score that deviation against explicit rules, and act only when the score justifies acting.

That pattern was implemented as a rule:based self:healing framework organised in five layers, with user:interface locator healing as its flagship capability. The framework records a Golden Fingerprint of each tracked element, and when a locator fails it extracts the live document structure in a single pass, scores every candidate against that fingerprint using four weighted rules covering visible text, structural position, styling and neighbouring elements, and applies threshold logic to choose between automatic recovery, cautious recovery under review, and refusal. High:confidence repairs are written back into the automation source so that the fix is permanent rather than per:run. The same detect:score:act pattern was extended to server:side recovery actions and to browser:side anomaly reporting, and all three engines write to one store and one dashboard. The layer attaches to an existing Selenium suite at one of three integration levels, the deepest of which requires no change to the test code at all. No machine learning is used anywhere: every decision follows predefined, inspectable rules.

Measured against an unmodified test suite under a realistic front:end refactor, the framework took that suite from one passing test out of five to five out of five, recovering four failing tests without a single edit to the test code, at a mean cost of 29.1 milliseconds per recovery. Because a resolved lookup no longer exhausts its wait timeout, the suite also ran roughly seven times faster than the failing baseline 2.4 seconds against 16.6 which shows that locator rot costs teams slow builds as well as red ones. Every healing decision was recorded together with the confidence score and the individual rule contributions that produced it. Where the evidence was insufficient the framework declined to act, raising a critical alert and returning control to a human rather than risking an interaction with the

wrong element; where a locator failed repeatedly it was locked rather than retried indefinitely. An automated harness confirmed the decision logic across all three confidence bands, and the framework rebuilt its own baseline unaided after that baseline was deleted. The project's contribution is less the act of recovering a broken locator than the demonstration that such recovery can be made accountable. Because every decision reduces to a weighted score over four inspectable attributes, it is always possible to say not merely that the framework healed something, but exactly why it was confident enough to do so and, just as importantly, why on occasion it was not. The most valuable lesson of the work came from a failure rather than a success. An early version was found to be healing its own baseline:discovery scan and manufacturing matches for elements that were never on the page. That result reframed the central risk of the entire idea: a self:healing layer that fails to heal leaves an engineer with a visible error, which is merely inconvenient, whereas a layer that heals incorrectly leaves them with a passing test that proves nothing, which is far worse. Much of the framework's final design the safety gate, the suppression of healing during learning, the deliberate decision to treat an empty result list as an answer rather than a failure, and the repeated:failure loop guard exists to keep the system on the safe side of that line.

Within the scope of a student major project, the work demonstrates that established self:healing principles can be applied to a single web application in a form that is lightweight enough to implement, transparent enough to validate, and conservative enough to trust.

5.2 Future Enhancement

5.2.1 Limitations and Challenges Encountered

The framework as delivered has known boundaries, several of which were identified during development and testing rather than assumed at the outset.

- The reported accuracy is a count of scenarios rather than a statistic over a large sample, and a page carrying many visually similar elements would compress the currently wide margin between a genuine match and a refusal.
- Source rewriting is textual. The write:back that makes a repair permanent edits the automation script by pattern substitution. This is adequate for a controlled project and retains a backup of every file it touches, but it is not an approach that should be

pointed at a production repository, where an unintended substitution could alter code the framework never examined.

- The element:type gate is absolute. A candidate whose tag differs from the stored fingerprint is skipped outright, so a button refactored into a link heals to nothing even when its text, position and neighbours are unchanged. This was a deliberately conservative choice, but it rejects matches that the remaining evidence would justify.
- Baseline rescanning merges rather than replaces. Because a rescan adds to the stored fingerprints instead of superseding them, entries recorded from earlier builds can persist after the elements they describe are gone. No incorrect heal was traced to this during testing, but a stale fingerprint winning an intent match is a genuine failure mode rather than a theoretical one.

5.2.2 Proposed Improvements

Several improvements follow directly from the limitations above and would strengthen the framework without altering its rule:based character.

- Replace textual source rewriting with a locator repository. Tests would reference elements by logical name and the framework would update the repository entry rather than the test file itself, with direct source patching retained as an explicit opt:in. This removes the sharpest risk in the current design and matches the approach taken by established industry tooling, which makes it easier to defend as well as safer to operate.
- Replace the absolute element:type gate with a heavy scoring penalty, so that a tag change reduces confidence sharply very likely into the cautious band, where a human reviews it instead of eliminating an otherwise strong candidate outright.
- Make baseline rescanning replace rather than merge, and expire fingerprints that have not matched an element for a configurable number of runs, so that the registry cannot silently accumulate entries describing builds that no longer exist.
- Make the rule weights configurable and calibrate them empirically. The weights of Table 3.1 were chosen by reasoning about which attributes tend to survive a refactor, and the results support them, but a study measuring which attributes actually persist across real commit histories would place them on an evidential rather than an intuitive footing.

- Migrate the store to a relational database once the schema has stabilised, so that healing history can be queried and analysed over time rather than only inspected.

5.2.3 Additional Features and Modules

Beyond refining what already exists, several additions would broaden what the framework can be used for.

- Multi:page and multi:application baselines, with fingerprints namespaced by application and route, so that one installation can serve several suites without the risk of an element on one page matching a fingerprint recorded on another.
- Support for shadow document trees and nested frames, which are common in component:based front ends and currently fall outside the scope of the extraction pass.
- Adapters for other automation frameworks. The heuristic core does not depend on Selenium, so exposing it behind a small adapter interface would allow the same engine to serve Playwright or Cypress suites.
- A continuous:integration reporting artefact. A machine:readable summary emitted at the end of each run what healed, at what confidence, and what was refused would let a pipeline distinguish a build that passed cleanly from one that passed only because the framework intervened, and would let a team track locator decay over time as a maintenance signal in its own right.
- A broader browser:side telemetry set, extending client:side reporting beyond failed resources and runtime errors to performance regressions and failed network calls.
- An expanded infrastructure rule library, with recovery time and accuracy measured across a wider range of simulated conditions than the set demonstrated here.

5.2.4 Scalability, Integration and Wider Application

The immediate scalability question is concurrent test execution. The store is already safe for concurrent access across processes, but a suite running many browsers in parallel would benefit from a shared service holding the fingerprint baseline in memory rather than each worker reading it from disk, and a rolling retention policy on healing records would keep the dashboard responsive as history accumulates.

On integration, the framework is currently attached in:process to a single test run. Running it as a small service, with the test:side layer reduced to a thin client, would let several suites share one baseline and one healing history, and would make the dashboard a team:level view rather than a per:run one. Integration with issue tracking is a natural extension: a

refusal below the safety gate is precisely the event that should open a maintenance ticket automatically, since it is by definition the case that a human must resolve.

The detect:score:act pattern is not specific to web user interfaces. The same fingerprint:and:score approach applies to mobile test automation, where element identifiers are equally volatile, and to robotic process automation, which suffers from the same brittleness for the same underlying reason. A related direction is interface contract healing, where a response field is renamed but its type, position and content remain recognisable.

As a research direction, the most useful further work would be a controlled comparison between this rule:based approach and a learned one on the same fault set, measuring not only healing accuracy but also false:heal rate and explainability. A hybrid arrangement learned candidate ranking constrained by rule:based thresholds that retain the power of refusal would test whether the accuracy of a model can be obtained without surrendering the property that makes this framework safe: that it can always explain its decision, and is always willing to decline.

References

- [1] M. Sloman, "Policy driven management for distributed systems," *Journal of network and Systems Management*, vol. 2, no. 4, pp. 333:360, 1994.
- [2] J. Strassner, *Policy:Based Network Management*, San Francisco: Morgan Kaufmann, 2003.
- [3] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *IEEE Computer*, vol. 36, no. 1, pp. 41:50, 2003.
- [4] H. Paul, "Autonomic computing: IBM's perspective," IBM, 2001.
- [5] D. Garlan, S.:W. Cheng, A.:C. Huang, B. Schmerl and P. Steenkiste, "Self:repairing systems," in *Proceedings of the ACM SIGSOFT Workshop on Self:Healing Systems*, 2002.
- [6] L. Cherkasova, "Monitoring performance of web services," *IEEE Transactions on Services Computing*, 2009.
- [7] R. Murch, *Autonomic Computing*, IBM Press, 2004.
- [8] S. Hariri, "Autonomic computing: Research challenges," *Cluster Computing*, vol. 9, no. 4, pp. 347:349, 2006.
- [9] M. Salehie and L. Tahvildari, "Self:adaptive software," *ACM Transactions on Autonomous and Adaptive Systems*, vol. 4, no. 2, 2009.
- [10] J. Joyce, "Monitoring distributed systems," *ACM Transactions on Computer Systems*, 1987.